

Objektorientierte Programmierung (OOP) mit Java | Einführung

Konzept	Beschreibung	Technische Details
Abstraktion (Abstraction)	Abstraktion ist der Prozess, nur die relevanten Details zu zeigen und unnötige Informationen zu verbergen. Sie hilft, die Komplexität zu reduzieren und das Design zu vereinfachen.	Abstrakte Klassen und Interfaces ermöglichen die Definition von gemeinsamen Schnittstellen, ohne die Details der Implementierung anzugeben. Eine abstrakte Klasse kann sowohl abstrakte Methoden (Methoden ohne Implementierung) als auch konkrete Methoden (mit Implementierung) enthalten. Ein Interface definiert nur Methodensignaturen, und die Implementierung erfolgt in den Klassen, die das Interface implementieren.
Objekte (Objects)	Ein Objekt ist eine Instanz einer Klasse, das heißt, es ist ein konkretes Exemplar einer Klasse mit eigenen Attributen und Zuständen.	Objekte werden durch den <code>new</code> Operator erstellt. Jedes Objekt hat eine eindeutige Identität. Objekte können in verschiedenen Zuständen existieren, die durch die Felder der Klasse bestimmt werden. Objekte können Methoden aufrufen, um ihr Verhalten zu steuern. In Java werden Objekte im Heap-Speicher verwaltet.
Klassen (Classes)	Eine Klasse ist der Bauplan für Objekte. Sie definiert die Struktur (Felder) und das Verhalten (Methoden) von Objekten. Klassen bieten die Grundlage für die Kapselung von Daten und Funktionen.	Klassen sind in Java als <code>class</code> deklariert. Sie können Felder (Instanzvariablen) und Methoden enthalten. Klassen können Parameter (Konstruktoren) verwenden, um Objekte zu initialisieren. Zugriffskontrollen (<code>public</code> , <code>private</code> , <code>protected</code>) bestimmen die Sichtbarkeit von Klassen und ihren Mitgliedern. Sie können auch abstrakte Klassen und Interfaces definieren.

Konzept	Beschreibung	Technische Details
Kapselung (Encapsulation)	Kapselung ist der Prozess, bei dem die Implementierung eines Objekts vor äußeren Zugriffen geschützt wird, sodass nur über definierte Schnittstellen auf die Daten zugegriffen werden kann.	Felder werden in der Regel als <code>private</code> deklariert, um den direkten Zugriff zu verhindern. Öffentliche Methoden (Getter und Setter) ermöglichen den Zugriff auf private Felder. Kapselung hilft, die Integrität der Daten zu wahren und die Komplexität des Codes zu reduzieren. Die Trennung von Interface und Implementation folgt dem Prinzip der Informationsverbergung.
Vererbung (Inheritance)	Vererbung ermöglicht einer Klasse, Eigenschaften und Methoden einer anderen Klasse zu übernehmen. Sie fördert Wiederverwendbarkeit und Hierarchie in der Codebasis.	Vererbung erfolgt durch die <code>extends</code> -Klausel. Eine Unterklasse erbt alle öffentlichen und geschützten Mitglieder der Oberklasse, kann jedoch neue Mitglieder hinzufügen oder geerbte überschreiben. In Java kann eine Klasse nur eine andere Klasse direkt erben (Einzelvererbung). Vererbung unterstützt das Konzept der Hierarchie und Modularität.
Polymorphismus (Polymorphism)	Polymorphismus ermöglicht es, dass dieselbe Methode unterschiedliche Implementierungen haben kann, basierend auf dem Typ des Objekts, das sie aufruft. Dies erfolgt durch Overloading oder Overriding.	Method Overloading: Gleicher Methodenname, aber unterschiedliche Parameter. Method Overriding: Eine Methode in der Unterklasse ersetzt eine Methode der Oberklasse. Polymorphismus ermöglicht es, dass Methoden unterschiedliche Implementierungen zur Laufzeit aufrufen, basierend auf dem Typ des Objekts. Java unterstützt dynamischen Polymorphismus durch die Laufzeitbindung von Methodenaufrufen.
Assoziation (Association)	Assoziation beschreibt die Beziehung zwischen zwei Objekten. Es gibt unterschiedliche Arten der Assoziation: Einwegoder Zweiwegbeziehungen.	Assoziationen können durch Referenzen zwischen Objekten dargestellt werden (z. B. ein Objekt hat ein anderes Objekt als Feld). Die Kardinalität beschreibt die Anzahl der Objekte, die in einer Assoziation verbunden sind (z. B. 1:1, 1:n, n:m). Assoziationen sind häufig bei der Modellierung von realen Beziehungen zwischen Entitäten wie "Kunde" und "Bestellung" oder "Mitarbeiter" und "Projekt" zu finden.

Konzept	Beschreibung	Technische Details
Aggregation (Aggregation)	Aggregation ist eine spezielle Form der Assoziation, bei der ein Objekt Teil eines anderen Objekts ist, aber auch ohne das übergeordnete Objekt existieren kann.	Aggregation zeigt eine "Hat-ein"-Beziehung (z. B. ein "Auto" hat einen "Motor"). Aggregierte Objekte sind nicht unbedingt von der Lebensdauer des Aggregatobjekts abhängig. In Java wird Aggregation durch Referenzen auf andere Objekte dargestellt, die in Feldern oder Attributen gespeichert werden.
Komposition (Composition)	Komposition ist eine stärkere Form der Aggregation, bei der das Lebenszyklusmanagement des gesamten Objekts von dem übergeordneten Objekt übernommen wird.	Komposition zeigt eine "Besteht-aus"-Beziehung (z. B. ein "Haus" besteht aus "Zimmern"). Wenn das übergeordnete Objekt zerstört wird, werden auch die enthaltenen Objekte zerstört. Komposition stellt eine engere Bindung zwischen den beteiligten Objekten dar und ist oft eine präferierte Wahl, wenn eine dauerhafte Beziehung erforderlich ist.
Interfaces	Ein Interface definiert eine Sammlung abstrakter Methoden, die von einer Klasse implementiert werden müssen. Interfaces ermöglichen Polymorphismus und dienen der Abstraktion von Implementierungen.	Interfaces können mehrere abstrakte Methoden definieren, aber keine Implementierungen. Eine Klasse, die ein Interface implementiert, muss alle Methoden dieses Interfaces überschreiben. Java ermöglicht Mehrfachvererbung durch Interfaces, was mit der Klassenvererbung nicht möglich ist. Interfaces können auch Konstanten enthalten, die als <code>public static final</code> deklariert werden.
Abstrakte Klassen	Eine abstrakte Klasse ist eine Klasse, die nicht instanziiert werden kann und die abstrakte Methoden enthalten kann, die von den Unterklassen implementiert werden müssen.	Abstrakte Klassen können sowohl abstrakte als auch konkrete Methoden enthalten. Sie dienen als Basis für Klassen, die von ihnen erben und die abstrakten Methoden implementieren müssen. Abstrakte Klassen sind besonders nützlich, wenn eine gemeinsame Funktionalität bereitgestellt werden soll, die nicht direkt instanziiert werden kann.

OOP | Grundprinzipien

Prinzip	Beschreibung	Java-spezifische Konzepte/Techniken
Abstraktion (Abstraction)	Abstraktion bedeutet, dass nur die relevanten Details eines Objekts sichtbar gemacht werden und irrelevante Details verborgen bleiben.	Abstrakte Klassen mit <code>abstract</code> -Schlüsselwort Schnittstellen (Interfaces) mit dem <code>interface</code> -Schlüsselwort Verbergen von Details durch abstrakte Methoden und Interfaces
Kapselung (Encapsulation)	Das Prinzip, dass die Implementierungsdetails eines Objekts verborgen werden, um eine saubere Trennung zwischen Interface und Implementierung zu gewährleisten.	Verwendung von <code>private</code> und <code>protected</code> -Zugriffsmodifizierern Getter und Setter-Methoden zur Steuerung des Zugriffs auf Felder
Vererbung (Inheritance)	Erlaubt es einer Klasse, von einer anderen zu erben, wodurch sie Eigenschaften und Methoden der übergeordneten Klasse übernehmen kann.	Verwendung von <code>extends</code> , um eine Klasse zu erweitern Polymorphismus durch Überschreibung (<code>@Override</code>) Superklassenmethoden mit <code>super</code> aufrufen
Polymorphismus (Polymorphism)	Polymorphismus erlaubt es Objekten unterschiedlicher Klassen, auf gleiche Weise angesprochen zu werden, obwohl sie unterschiedliche Implementierungen haben.	Überladen von Methoden (<code>method overloading</code>) Überschreiben von Methoden (<code>method overriding</code>) Dynamische Bindung und späte Bindung bei Methodenaufrufen
Konstruktoren (Constructors)	Konstruktoren sind spezielle Methoden, die beim Erstellen eines Objekts aufgerufen werden, um das Objekt zu initialisieren.	Konstruktoren mit dem gleichen Namen wie die Klasse Überladen von Konstruktoren (Multiple Konstruktoren in einer Klasse)
Interfaces	Interfaces definieren Verträge, die von Klassen implementiert werden müssen.	<code>interface</code> -Schlüsselwort für das Erstellen von Interfaces Implementierung von Interfaces in Klassen mit <code>implements</code> Mehrfachvererbung durch Interfaces

Prinzip	Beschreibung	Java-spezifische Konzepte/Techniken
Abstrakte Klassen	Abstrakte Klassen sind Klassen, die nicht direkt instanziiert werden können und oft als Basis für andere Klassen dienen.	Verwendung des <code>abstract</code> -Schlüsselworts für abstrakte Klassen Abstrakte Methoden ohne Implementierung Instanziierung nur über abgeleitete Klassen
Methodenüberladung (Method Overloading)	Methoden mit dem gleichen Namen, aber unterschiedlichen Parameterlisten innerhalb einer Klasse.	Unterschiedliche Parameteranzahl oder -typen bei Methoden mit gleichem Namen
Methodenüberschreibung (Method Overriding)	Das Überschreiben von Methoden aus der Basisklasse, um eine spezifische Implementierung in der abgeleiteten Klasse bereitzustellen.	Verwendung des <code>@Override</code> -Annotations Anpassen der Methode in einer Subklasse, um das Verhalten zu ändern
Komposition (Composition)	Eine Klasse enthält Instanzen anderer Klassen als Mitglieder (besteht aus Objekten anderer Klassen).	Instanziierung von Objekten innerhalb einer Klasse und deren Verwendung als Felder
Assoziationen (Associations)	Beschreibt die Beziehungen zwischen Klassen, etwa „hat-ein“ oder „ist-ein“ Beziehung.	Eine Klasse kann Objekte anderer Klassen referenzieren. Verwendung von Aggregation und Komposition als stärkere Assoziationen
Verborgene Implementierung (Hiding Implementation)	Trennung von API und Implementierung, sodass Änderungen an der Implementierung die Benutzer des Objekts nicht beeinflussen.	Internes Verstecken von Feldern und Methoden durch <code>private</code> Bereitstellung öffentlicher Methoden (API), die auf interne Logik zugreifen
Static- und Instanzmethoden	Unterscheidung zwischen Methoden, die auf Instanzobjekte angewendet werden und Methoden, die auf die Klasse selbst angewendet werden.	<code>static</code> für Methoden, die ohne Instanziierung der Klasse aufgerufen werden können Instanzmethoden, die spezifisch für Objekte sind
Generics (Generische Typen)	Generics ermöglichen es, Klassen, Interfaces und Methoden mit Platzhaltern für Datentypen zu definieren.	Verwendung von <code><></code> -Platzhaltern, um Typen zur Laufzeit festzulegen (z.B. <code>List<T></code>)

Prinzip	Beschreibung	Java-spezifische Konzepte/Techniken
Enums	Enums definieren eine feste Menge von konstanten Werten.	Verwendung des <code>enum</code> -Schlüsselworts, um eine Aufzählung von Konstanten zu erstellen
Innere Klassen (Inner Classes)	Eine Klasse kann innerhalb einer anderen Klasse definiert werden, um den Zugriff auf die äußere Klasse zu erleichtern.	Statische und nicht-statische innere Klassen (mit <code>static</code> für statische innere Klassen) Anonyme und lokale innere Klassen

Abstraktion

Aspekt	Beschreibung
Definition	Abstraktion ist das Prinzip, nur die wesentlichen Merkmale eines Objekts bereitzustellen, während die komplexen Implementierungsdetails verborgen bleiben.
Ziel	Reduzierung der Komplexität und Bereitstellung einer klaren Schnittstelle für den Benutzer, ohne interne Details preiszugeben.
Umsetzung in Java	Wird durch abstrakte Klassen und Interfaces realisiert.
Abstrakte Klasse	Eine Klasse, die als Blaupause für Subklassen dient und mindestens eine abstrakte Methode enthalten kann. Kann auch konkrete Methoden und Felder haben.
Interface	Eine rein abstrakte Definition von Methoden (nur Methodensignaturen). Eine Klasse, die ein Interface implementiert, muss alle Methoden implementieren.
Schlüsselwörter	<code>abstract</code> für abstrakte Klassen und Methoden <code>interface</code> für Interfaces <code>implements</code> zur Implementierung eines Interfaces in einer Klasse <code>extends</code> zur Vererbung von abstrakten Klassen
Instanziierung	Eine abstrakte Klasse kann nicht direkt instanziiert werden, sondern nur über eine Subklasse. Ein Interface kann nicht instanziiert werden, sondern nur von einer Klasse implementiert werden.
Methoden in abstrakten Klassen	Können sowohl abstrakte Methoden (ohne Implementierung) als auch konkrete Methoden (mit Implementierung) enthalten.
Methoden in Interfaces	Vor Java 8: Nur abstrakte Methoden (keine Implementierung). Seit Java 8: Default-Methoden mit Implementierung möglich (<code>default</code> -Schlüsselwort). Seit Java 9: Private Methoden innerhalb eines Interfaces möglich.
Konstruktoren	Abstrakte Klassen dürfen Konstruktoren haben, um gemeinsame Eigenschaften für Subklassen zu initialisieren. Interfaces dürfen keine Konstruktoren haben.
Zugriffsmodifikatoren in abstrakten Klassen	Können <code>private</code> , <code>protected</code> , <code>public</code> oder <code>package-private</code> (<code>default</code>) Methoden und Felder enthalten.
Zugriffsmodifikatoren in Interfaces	Methoden in Interfaces sind standardmäßig <code>public</code> . Felder sind standardmäßig <code>public static final</code> (Konstanten).
Mehrfachvererbung	Java erlaubt keine Mehrfachvererbung von Klassen (d. h. eine Klasse kann nur eine andere Klasse <code>extends</code>). Eine Klasse kann jedoch mehrere Interfaces implementieren (<code>implements Interface01, Interface02</code>).

Aspekt	Beschreibung
Unterschiede zu Kapselung	Abstraktion konzentriert sich auf "Was tut ein Objekt?", nicht auf "Wie wird es umgesetzt?". Kapselung verbirgt die internen Daten eines Objekts und stellt nur kontrollierten Zugriff bereit.
Vorteile	Erhöhte Modularität: Erlaubt die Trennung von Schnittstelle und Implementierung. Erleichtert Wartung und Erweiterung: Implementierungsdetails können geändert werden, ohne die Schnittstelle zu beeinflussen. Fördert Wiederverwendbarkeit: Allgemeine Funktionalität kann über abstrakte Klassen oder Interfaces bereitgestellt werden.
Nachteile	Höherer Implementierungsaufwand: Erfordert zusätzliche Klassen zur konkreten Implementierung. Kann Code-Komplexität erhöhen: Zu viele Abstraktionen können das Verständnis erschweren.

Abstraktion

Kriterium	Beschreibung	Wann zu verwenden?
Definition von Abstraktion	Abstraktion bedeutet, dass nur die wesentlichen Eigenschaften und Verhaltensweisen eines Objekts dargestellt werden, während unwichtige Details verborgen bleiben.	Wenn eine Klasse oder ein Objekt nur auf eine bestimmte Weise verwendet werden soll, ohne sich mit den Details der Implementierung zu befassen.
Verwendung von abstrakten Klassen	Eine abstrakte Klasse ist eine Klasse, die nicht instanziiert werden kann und als Basis für andere Klassen dient. Sie kann sowohl abstrakte als auch konkrete Methoden enthalten.	Wenn eine allgemeine Klasse bereitgestellt werden soll, die von anderen Klassen erweitert werden muss, aber keine vollständige Implementierung bietet.
Verwendung von Schnittstellen	Eine Schnittstelle definiert nur die Signaturen von Methoden, ohne eine konkrete Implementierung zu bieten. Sie dient als Vertrag für die Klassen, die sie implementieren.	Wenn eine Gruppe von Klassen dasselbe Verhalten aufweisen soll, aber unterschiedliche Implementierungen benötigen.
Ziel der Abstraktion	Abstraktion zielt darauf ab, die Komplexität zu verringern, indem nur die relevanten Details präsentiert werden. Dies ermöglicht eine klare Trennung zwischen den "Was" und den "Wie".	Wenn die Implementierungsdetails nicht von der restlichen Software gesehen werden sollen, sondern nur die Interaktion mit einem Objekt von Interesse ist.
Verhinderung von Überinformation	Abstraktion hilft, den Code übersichtlicher und verständlicher zu machen, indem sie nur die wesentlichen Eigenschaften und Methoden zeigt.	Wenn unnötige Details vom Benutzer oder anderen Klassen verborgen werden sollen, um die Nutzung zu vereinfachen.
Förderung der Wiederverwendbarkeit	Abstraktion ermöglicht es, allgemeine Funktionalitäten in abstrakten Klassen oder Schnittstellen zu definieren, die von verschiedenen konkreten Implementierungen genutzt werden können.	Wenn dieselbe Logik oder Funktionalität in verschiedenen Klassen benötigt wird, aber die Implementierung unterschiedlich sein soll.

Kriterium	Beschreibung	Wann zu verwenden?
Unterschied zu Kapselung	Während Kapselung dazu dient, Details einer Klasse zu verbergen und den Zugriff zu steuern, geht es bei der Abstraktion darum, den Fokus auf das Wesentliche zu lenken und unwichtige Details auszublenden.	Wenn der Fokus auf der Struktur und den gemeinsamen Funktionen von Objekten liegt, ohne sich mit den Details ihrer Implementierung zu befassen.
Vermeidung von engen Kopplungen	Durch Abstraktion können Klassen und Komponenten so gestaltet werden, dass sie nur über Schnittstellen oder abstrakte Klassen miteinander interagieren, anstatt direkt auf konkrete Implementierungen zuzugreifen.	Wenn Klassen und Komponenten unabhängig voneinander entwickelt und getestet werden sollen.
Erhöhung der Flexibilität	Abstraktion ermöglicht es, das Verhalten von Objekten zur Laufzeit zu ändern oder auszutauschen, ohne dass andere Teile des Systems beeinträchtigt werden.	Wenn ein System oder eine Komponente flexibel und leicht erweiterbar sein soll.
Verwendung von abstrakten Methoden	Abstrakte Methoden in abstrakten Klassen und Schnittstellen sind Methoden, die von den abgeleiteten Klassen implementiert werden müssen, aber keine eigene Implementierung in der abstrakten Klasse haben.	Wenn eine allgemeine Schnittstelle bereitgestellt werden soll, aber die konkrete Implementierung in den abgeleiteten Klassen erfolgt.
Zugriff auf konkrete Implementierungen	Abstraktion trennt die Darstellung der Funktionalität von ihrer konkreten Implementierung, sodass der Benutzer nur mit den abstrahierten Methoden interagiert, ohne die Implementierung zu kennen.	Wenn nur die Funktionalität und nicht die konkrete Implementierung eines Objekts benötigt wird, z. B. beim Arbeiten mit Schnittstellen.
Verwendung bei Schnittstellen	Schnittstellen bieten eine Form der Abstraktion, indem sie nur die Signaturen von Methoden definieren, die von der implementierenden Klasse bereitgestellt werden müssen.	Wenn ein Verhalten in verschiedenen Klassen definiert werden soll, aber die Implementierungen flexibel bleiben müssen.

Kriterium	Beschreibung	Wann zu verwenden?
Ermöglichung der Erweiterbarkeit	Abstraktion sorgt dafür, dass das System leicht erweiterbar bleibt, da neue konkrete Klassen hinzugefügt werden können, ohne bestehende Klassen zu verändern.	Wenn das System zukünftige Erweiterungen und neue Funktionalitäten unterstützen soll.
Abstraktion und Polymorphismus	Abstraktion in Verbindung mit Polymorphismus ermöglicht es, dass Objekte verschiedener konkreter Typen über eine gemeinsame Schnittstelle oder abstrakte Klasse referenziert und verwendet werden können.	Wenn Objekte verschiedener Klassen auf die gleiche Weise behandelt werden sollen, ohne ihre konkreten Typen zu kennen.

Zusammenfassung

Abstraktion ist ein grundlegendes Konzept in der objektorientierten Programmierung, das darauf abzielt, die Komplexität zu verringern, indem nur die wesentlichen Details einer Klasse oder eines Objekts sichtbar gemacht werden, während die weniger wichtigen Implementierungsdetails verborgen bleiben. Abstraktion wird durch abstrakte Klassen und Schnittstellen erreicht, die die Grundlage für verschiedene Implementierungen bieten und Flexibilität sowie Erweiterbarkeit ermöglichen.

Wann verwenden?

- Wenn die Implementierungsdetails eines Objekts nicht für den Benutzer sichtbar sein sollen, sondern nur die wesentlichen Eigenschaften und Verhaltensweisen.
- Wenn mehrere Klassen ein ähnliches Verhalten aufweisen sollen, jedoch unterschiedliche Implementierungen haben.
- Wenn eine gemeinsame Struktur oder Funktionalität bereitgestellt werden soll, die von verschiedenen konkreten Klassen implementiert werden kann.

Vorteile

- **Verringert Komplexität**
Nur die relevanten Details sind sichtbar, was das Verständnis und die Nutzung des Codes vereinfacht.
- **Erhöht Flexibilität und Erweiterbarkeit**
Neue Implementierungen können hinzugefügt werden, ohne bestehende Implementierungen zu verändern.
- **Fördert Wiederverwendbarkeit**
Abstrakte Klassen und Schnittstellen ermöglichen es, eine allgemeine Struktur zu definieren, die von mehreren Klassen verwendet werden kann.

Nachteile

- **Overhead**
Die Verwendung von abstrakten Klassen und Schnittstellen kann zusätzlichen Aufwand und Komplexität in der Implementierung verursachen.

- **Verborgene Implementierungsdetails**

Während Abstraktion eine nützliche Vereinfachung darstellt, kann sie auch dazu führen, dass der Entwickler die zugrunde liegende Implementierung nicht versteht.

Airbus A350-900 | Abstraktion

Kategorie	Beschreibung
Definition von Abstraktion	Abstraktion ist das Prinzip der objektorientierten Programmierung, bei dem nur die wesentlichen Merkmale eines Objekts dargestellt werden, während unwichtige Details verborgen bleiben. Sie ermöglicht die Modellierung komplexer Systeme auf einer höheren, verständlicheren Ebene.
Anwendungsfall (Airbus A350-900)	Bei der Modellierung eines Airbus A350-900 werden komplexe Systeme wie Engine , FlightControlSystem und Avionics abstrahiert, um die Interaktionen auf einer höheren Ebene darzustellen, die für die Steuerung und Verwaltung des Flugzeugs relevant sind.
Abstraktionsebenen	Das Modell des Airbus A350-900 kann in mehrere abstrakte Ebenen unterteilt werden, um verschiedene Teile des Flugzeugs zu isolieren und damit den Zugriff auf relevante Details zu vereinfachen.
Höchste Abstraktionsebene	Auf dieser Ebene wird der Airbus A350-900 als gesamtes Objekt betrachtet, mit Eigenschaften wie Flugzeugkennung, Flugstatus, Reichweite und aktuellen Betriebsparametern, ohne auf die Details einzelner Systeme einzugehen.
Mittlere Abstraktionsebene	Das Flugzeug wird in wesentliche Teilsysteme unterteilt. Jedes System ist ein abstraktes Modell eines komplexeren, realen Systems, wie zum Beispiel: FlightControlSystem , Engine , Avionics , LandingGear , Cabin , FuelSystem .
Niedrigste Abstraktionsebene	Auf dieser Ebene werden spezifische Komponenten innerhalb eines Systems abstrahiert. Zum Beispiel besteht das Engine -System aus vielen Komponenten wie Compressor , Turbine , FuelInjector , ThrustControl , die jeweils über eigene Eigenschaften und Methoden verfügen.
Abstraktion durch Schnittstellen	Schnittstellen definieren das Verhalten und die Interaktion zwischen verschiedenen Komponenten, ohne Details zur Implementierung zu offenbaren. Ein ControlSystemInterface könnte Methoden wie <code>adjustAltitude()</code> , <code>adjustSpeed()</code> , <code>correctCourse()</code> bereitstellen, während das spezifische FlyByWireSystem diese Methoden im Detail umsetzt.
Abstraktion durch abstrakte Klassen	Abstrakte Klassen bieten eine Basisklasse mit gemeinsamen Eigenschaften und Methoden für verwandte Objekte. Zum Beispiel könnte die abstrakte Klasse AircraftSystem grundlegende Funktionen wie <code>start()</code> und <code>shutdown()</code> definieren, während die spezifischen Systeme wie Engine und Avionics diese Methoden implementieren.

Kategorie	Beschreibung
Beispiel: Triebwerksabstraktion	Anstatt jedes Engine -Modell einzeln zu implementieren, könnte eine abstrakte Klasse Engine Methoden wie <code>start()</code> , <code>stop()</code> , <code>increaseThrust()</code> enthalten, während spezifische Triebwerke wie RollsRoyceTrentXWB diese Methoden mit ihren eigenen Parametern und Details erweitern.
Abstraktion im Flugsteuerungssystem	Ein allgemeines FlightControlSystem könnte Methoden wie <code>adjustAltitude()</code> oder <code>changeDirection()</code> abstrahieren, ohne sich um die spezifische Implementierung von FlyByWire oder HydraulicSystems zu kümmern. Diese Systeme würden dann die allgemeinen Methoden implementieren, je nach ihrer Funktionsweise.
Abstraktion im Treibstoffsystem	Das FuelSystem könnte in einer abstrakten Klasse grundlegende Funktionen wie <code>checkFuelLevel()</code> , <code>fuelConsumption()</code> und <code>refuel()</code> definieren, während spezielle FuelPump -Komponenten diese auf ihre eigene Weise realisieren.
Vorteile der Abstraktion	Komplexitätsreduktion: Nur relevante Details werden gezeigt, wodurch die Komplexität des gesamten Systems verringert wird. Wartbarkeit: Änderungen an spezifischen Implementierungen wirken sich nicht direkt auf die gesamte Systemstruktur aus. Wiederverwendbarkeit: Abstrakte Modelle können leicht in verschiedenen Systemen wiederverwendet werden. Erweiterbarkeit: Neue Komponenten oder Systeme können ohne großen Einfluss auf das bestehende System hinzugefügt werden.
Zusammenhang mit anderen OOP-Prinzipien	Kapselung (Encapsulation): Abstraktion verbirgt die Implementierungsdetails und schützt so vor direktem Zugriff. Vererbung (Inheritance): Abstraktion nutzt Vererbung, um spezifische Implementierungen von Basisklassen zu erben und zu erweitern. Polymorphie (Polymorphism): Verschiedene konkrete Implementierungen von abstrakten Klassen oder Interfaces können austauschbar verwendet werden, je nachdem, welche konkreten Objekte zum Laufzeitzeitpunkt erstellt werden.

Kapselung (Encapsulation)

Kriterium	Beschreibung	Wann zu verwenden?
Definition von Kapselung	Kapselung bedeutet in der objektorientierten Programmierung, dass die inneren Details einer Klasse (Daten und Implementierungen) verborgen werden, und nur öffentliche Methoden den Zugriff darauf ermöglichen.	Wenn der Zugriff auf die inneren Daten einer Klasse kontrolliert und nur über explizite Methoden erfolgen soll.
Datenverbergen	Kapselung schützt die Daten einer Klasse, indem sie in privaten Feldern (<code>private</code>) gespeichert werden, um direkten Zugriff von außen zu verhindern.	Wenn Daten vor unbefugtem Zugriff oder unerwünschten Änderungen geschützt werden sollen.
Zugriffskontrolle über Getter und Setter	Anstatt direkt auf Felder einer Klasse zuzugreifen, werden spezielle Methoden (Getter/Setter) bereitgestellt, um auf private Felder zuzugreifen oder sie zu verändern.	Wenn der Zugriff auf Felder kontrolliert werden muss, z. B. durch Validierung oder das Setzen von Bedingungen.
Erhöhung der Wartbarkeit	Durch Kapselung wird die Wartbarkeit des Codes erhöht, da die inneren Implementierungsdetails von anderen Klassen nicht direkt verändert werden können. Änderungen können lokal innerhalb der Klasse vorgenommen werden.	Wenn eine Klasse von außen nicht direkt beeinflusst werden soll und Änderungen an der Implementierung nicht den gesamten Code brechen sollen.
Sichtbarkeitsmodifizierer	Kapselung wird durch die Verwendung von Zugriffsmodifizierern wie <code>private</code> , <code>protected</code> und <code>public</code> erreicht, um zu definieren, welche Teile der Klasse von außen zugänglich sind.	Wenn der Zugriff auf Methoden oder Felder durch Sichtbarkeitsmodifizierer geschützt und gezielt gestaltet werden soll.
Vermeidung von Abhängigkeiten	Kapselung hilft dabei, die Abhängigkeiten zwischen den Klassen zu reduzieren. Durch die Bereitstellung von kontrolliertem Zugriff auf Daten werden unerwünschte Verbindungen verhindert.	Wenn eine enge Kopplung zwischen den Klassen vermieden werden soll und nur notwendiger Zugriff auf die Daten gewährt werden soll.

Kriterium	Beschreibung	Wann zu verwenden?
Verantwortlichkeit von Klassen	Kapselung sorgt dafür, dass eine Klasse nur für ihr eigenes Verhalten und ihre eigenen Daten verantwortlich ist und keine Informationen von außen preisgibt.	Wenn eine Klasse ihre Daten und ihr Verhalten klar von anderen Klassen trennen soll, um Verantwortlichkeiten zu definieren.
Sichtbarkeit von Feldern	Felder einer Klasse sollten möglichst privat sein, um sicherzustellen, dass sie nicht versehentlich oder unerwünscht von außen verändert werden können.	Wenn Daten nicht direkt von außen zugänglich sein sollen, sondern nur durch gezielte Methoden manipulierbar sind.
Verwendung von <code>final</code> bei Feldern	Felder, die nicht verändert werden sollen, können als <code>final</code> deklariert werden, um sie nach der Initialisierung konstant zu machen.	Wenn Felder der Klasse einmal gesetzt und nicht mehr verändert werden sollen.
Schutz vor fehlerhaften Eingaben	Getter- und Setter-Methoden bieten eine Möglichkeit, Eingabewerte zu validieren, bevor sie die privaten Felder ändern.	Wenn sichergestellt werden soll, dass Felder nur gültige Werte erhalten, um Fehler oder Inkonsistenzen zu vermeiden.
Erleichterung der Testbarkeit	Durch Kapselung kann das Testen von Klassen erleichtert werden, da nur die öffentlichen Methoden getestet werden müssen und die Implementierung intern geändert werden kann, ohne dass bestehende Tests angepasst werden müssen.	Wenn die Tests einer Klasse von den internen Details entkoppelt werden sollen.
Verhinderung von direktem Zugriff auf komplexe Datenstrukturen	Kapselung verhindert den direkten Zugriff auf komplexe Datenstrukturen, indem die Daten intern als private Felder verwaltet und nur über kontrollierte Methoden zugänglich gemacht werden.	Wenn der direkte Zugriff auf komplexe interne Datenstrukturen verhindert werden soll, um unvorhersehbare oder fehlerhafte Manipulationen zu vermeiden.
Verwendung von Abstraktion	Durch Kapselung wird eine Abstraktionsschicht geschaffen, die den Benutzer vor den internen Details der Klasse verbirgt und nur die relevanten Informationen bereitstellt.	Wenn eine Klasse ein höheres Maß an Abstraktion erfordert, bei dem der Benutzer nur mit den wichtigsten Methoden interagiert.

Kriterium	Beschreibung	Wann zu verwenden?
Verhindern von zufälligen Modifikationen	Kapselung schützt vor zufälligen oder unerwünschten Änderungen durch die Kontrolle, wie Daten in einer Klasse geändert werden können.	Wenn Änderungen an Daten nur durch explizit definierte Wege und Bedingungen erfolgen sollen.

Zusammenfassung

Kapselung ist eines der grundlegenden Prinzipien der objektorientierten Programmierung und dient dazu, die Implementierungsdetails einer Klasse zu verbergen und nur bestimmte Daten und Methoden öffentlich zugänglich zu machen. Sie schützt die Daten der Klasse vor unbefugtem Zugriff und Manipulation und ermöglicht so eine bessere Kontrolle, Wartbarkeit und Flexibilität.

Wann verwenden?

- Wenn die Daten einer Klasse nicht direkt zugänglich gemacht werden sollen, um sie vor unerwünschten Änderungen zu schützen.
- Wenn der Zugriff auf Felder und Methoden kontrolliert und validiert werden soll.
- Wenn die Implementierungsdetails der Klasse von außen verborgen werden sollen, um den Code flexibler und wartungsfreundlicher zu gestalten.

Vorteile

- **Datenkapselung**
Schützt die Daten der Klasse vor unbefugtem Zugriff und Manipulation.
- **Kontrollierter Zugriff**
Ermöglicht es, den Zugriff auf private Felder und Methoden nur durch explizit definierte Methoden zu steuern.
- **Wartbarkeit und Flexibilität**
Durch Kapselung können Änderungen an der internen Implementierung vorgenommen werden, ohne dass der äußere Code davon betroffen ist.
- **Fehlervermeidung**
Verhindert unvorhersehbare Fehler durch direkte Manipulation von Feldern, insbesondere durch Eingabevalidierungen.

Nachteile

- **Overhead**
Die Implementierung von Getter- und Setter-Methoden für jedes Feld kann zusätzlichen Code und Aufwand verursachen.
- **Zugriffsbarrieren**
Bei einer übermäßigen Kapselung kann es schwieriger werden, auf Daten zuzugreifen, was die Lesbarkeit und Verständlichkeit des Codes beeinträchtigen kann.

Kapselung (Encapsulation)

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Definition	Kapselung ist ein Grundprinzip der objektorientierten Programmierung (OOP), das den direkten Zugriff auf interne Daten einer Klasse einschränkt.	Nur über definierte Methoden (Getter/Setter oder Business-Methoden) kann auf die Daten zugegriffen werden. Verbessert die Sicherheit und Wartbarkeit des Codes.
Schutz interner Zustände	Durch Kapselung wird verhindert, dass Variablen oder Zustände direkt von außen verändert oder ausgelesen werden können.	Zugriffskontrolle erfolgt durch private Felder und öffentliche Methoden zur Datenmanipulation. Schutz vor unerwarteten Zustandsänderungen und Inkonsistenzen.
Zugriffsmodifizierer	Java bietet verschiedene Modifizierer (<code>private</code> , <code>protected</code> , <code>public</code> , <code>package-private</code>), um den Zugriff auf Attribute und Methoden zu kontrollieren.	<code>private</code> : Zugriff nur innerhalb der Klasse. <code>protected</code> : Zugriff innerhalb des Pakets und durch Unterklassen. <code>public</code> : Vollständiger Zugriff. Kein Modifizierer: Paketzugriff.
Datenhiding (Datenverbergung)	Das Verbergen von Implementierungsdetails verhindert ungewollte Abhängigkeiten und erleichtert Änderungen an der internen Implementierung.	Innere Datenstrukturen können geändert werden, ohne dass andere Klassen betroffen sind. Erhöht die Robustheit, indem der externe Zugriff begrenzt wird.
Getter und Setter	Bereitstellung kontrollierter Zugriffsmethoden anstelle direkter Feldmanipulation zur Kontrolle von Lese- und Schreibzugriffen.	<code>getX()</code> für das Lesen von Attributen. <code>setX(value)</code> für das Schreiben mit optionaler Validierung. Ermöglicht Berechnungen oder Transformationen beim Zugriff.
Unveränderlichkeit (Immutability)	Durch Kapselung kann ein Objekt so gestaltet werden, dass es nach der Initialisierung nicht mehr verändert werden kann.	Vermeidung von Settern für unveränderliche Felder. Initialisierung durch Konstruktoren. Verwendung von <code>final</code> für Variablen.
Final-Modifikator für Sicherheit	<code>final</code> kann verwendet werden, um sicherzustellen, dass Referenzen oder Methoden nicht unerwartet verändert oder überschrieben werden.	<code>final</code> für Felder bedeutet, dass sie nach der Initialisierung nicht geändert werden können. <code>final</code> Methoden können nicht von Unterklassen überschrieben werden.

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Encapsulation und Konstruktoren	Konstruktoren dienen zur Initialisierung von Objekten und können mit spezifischen Einschränkungen versehen werden, um Konsistenz zu gewährleisten.	Pflichtparameter können über Konstruktoren gesetzt werden. Verhindert nicht zulässige Objektzustände. Konstruktor-Überladung für verschiedene Initialisierungsszenarien.
Datenvalidierung	Kapselung ermöglicht Validierungen in den Settermethoden, um ungültige Zustände zu verhindern.	Beispiel: Ein <code>setAge(int age)</code> könnte sicherstellen, dass <code>age > 0</code> ist. Verhindert fehlerhafte Eingaben durch externe Klassen.
Thread-Sicherheit durch Kapselung	Durch das Verbergen von Zuständen und die Bereitstellung synchronisierter Zugriffsmethoden kann Thread-Sicherheit verbessert werden.	Synchronisierte Methoden (<code>synchronized</code>) oder <code>volatile</code> für sichere nebenläufige Zugriffe. Reduktion von Race Conditions durch private Felder und kontrollierten Zugriff.
Encapsulation und Datenkapselung mit Records	Java 14 führte <code>record</code> -Klassen ein, die standardmäßig Felder als <code>private final</code> kapseln und automatische Getter bereitstellen.	<code>record Person(String name, int age) {}</code> generiert automatisch <code>private final</code> Felder mit öffentlichen Getter-Methoden. Reduziert Boilerplate-Code und fördert Unveränderlichkeit.
Verhinderung direkter Objektmanipulation	Eine Klasse kann Methoden bereitstellen, die intern bestimmte Abläufe durchführen, anstatt externe Manipulation zuzulassen.	Direkte Feldänderungen werden vermieden. Beispiel: <code>account.deposit(amount)</code> statt <code>account.balance += amount</code> .
Kapselung in APIs	Public APIs kapseln interne Implementierungsdetails, um Änderungen zu erleichtern und Kompatibilität zu bewahren.	Interne Implementierungsdetails sind versteckt. Die API bleibt stabil, auch wenn sich die interne Logik ändert.
Kapselung und Vererbung	Kapselung beeinflusst die Sichtbarkeit von geerbten Eigenschaften und Methoden.	<code>private</code> Mitglieder der Superklasse sind in der Unterklasse nicht sichtbar. <code>protected</code> erlaubt Vererbung, aber nicht öffentlichen Zugriff.
Encapsulation und JavaBeans-Konvention	JavaBeans sind Klassen, die private Felder und öffentliche Getter/Setter bereitstellen, um Kapselung zu gewährleisten.	Beispiel: <code>private String name;</code> <code>public String getName() { return name; }</code> <code>public void setName(String name) { this.name = name; }</code>

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Reduzierung von Abhängigkeiten	Durch das Verstecken interner Implementierungsdetails wird die Kopplung zwischen Klassen reduziert.	Änderungen an einer Klasse haben weniger Auswirkungen auf andere Klassen. Förderung des Low Coupling & High Cohesion-Prinzips.
Encapsulation und das SOLID-Prinzip	Kapselung unterstützt das Single Responsibility Principle (SRP) und das Open/Closed Principle (OCP) aus den SOLID-Prinzipien.	SRP: Jedes Objekt sollte nur für seine eigenen Daten verantwortlich sein. OCP: Interne Implementierungen können geändert werden, ohne andere Klassen zu beeinflussen.
Encapsulation und Sicherheitsaspekte	Sensible Daten können vor unberechtigtem Zugriff geschützt werden, indem sie gekapselt und nur über kontrollierte Mechanismen zugänglich gemacht werden.	Beispiel: Passwort-Hashing in einer Methode statt direkter Speicherung von Passwörtern. Begrenzung der Sichtbarkeit von sicherheitskritischen Methoden.
Encapsulation und Refactoring	Kapselung erleichtert das Refactoring, da Änderungen an internen Implementierungsdetails keine Auswirkungen auf externe Klassen haben.	Methoden können ohne Änderung externer Klassen umstrukturiert werden. Verbesserungen wie Caching oder Logging können hinzugefügt werden, ohne die öffentliche API zu verändern.

Kapselung in Java ist ein fundamentales Prinzip, das sicherstellt, dass die interne Struktur eines Objekts vor äußeren Zugriffen geschützt ist. Nur vordefinierte Schnittstellen (Methoden) sind zugänglich.

- **Zugriffsmodifikatoren** in Java steuern den Zugriff auf Felder und Methoden:
 - **private:** Nur innerhalb der Klasse zugänglich.
 - **protected:** Innerhalb der Klasse, des Pakets und in Unterklassen zugänglich.
 - **public:** Überall zugänglich.
 - **default (Package-Private):** Nur innerhalb des gleichen Pakets zugänglich.
- **Getter- und Setter-Methoden:** Dies sind Methoden, die den Zugriff auf private Felder ermöglichen. Sie kontrollieren den Zugriff auf Daten und verhindern, dass diese direkt verändert werden, was zu unvorhersehbarem Verhalten führen könnte. Getter sind oft einfache Methoden, die den Wert eines privaten Feldes zurückgeben, während Setter die Möglichkeit bieten, den Wert zu setzen und Validierungslogik anzuwenden.
- **Datenvalidierung** innerhalb von Setter-Methoden stellt sicher, dass nur gültige Daten einem Objekt zugewiesen werden. Dies erhöht die Datenintegrität und schützt vor fehlerhaften Zuständen.

Zusammenfassung

Kapselung ist ein essenzielles Konzept in Java, das den Zugriff auf interne Daten schützt, die Sicherheit und Wartbarkeit von Code verbessert und eine klare Trennung zwischen Implementierung und Schnittstelle ermöglicht. Sie hilft, fehlerhafte Zustände zu vermeiden, ermöglicht eine kontrollierte Datenmanipulation und reduziert Abhängigkeiten zwischen Klassen.

Prof. Dr. Carsten Müller | Software Engineering

Airbus A350-900 | Kapselung

Kategorie	Beschreibung
Definition von Kapselung	Kapselung ist das OOP-Prinzip, bei dem die internen Details eines Objekts verborgen werden und nur notwendige Informationen über öffentliche Schnittstellen zugänglich sind. Dies schützt die Integrität des Objekts und verhindert unerwünschte Veränderungen durch externe Akteure.
Anwendungsfall (Airbus A350-900)	Bei der Modellierung eines Airbus A350-900 werden interne Details jedes Systems wie Engine , FuelSystem oder FlightControlSystem verborgen. Die Systeme bieten nur öffentliche Schnittstellen für den Zugriff auf die wesentlichen Funktionen, was das Risiko von fehlerhaften Eingriffen verringert und die Wartbarkeit verbessert.
Ziele der Kapselung	Datenversteckung: Verbergen von internen Zuständen und Implementierungsdetails. Schutz der Objektintegrität: Verhindern von unzulässigen Änderungen durch externe Einflüsse. Modularisierung: Das System wird in klar abgegrenzte, selbstgenügsame Teile unterteilt.
Beispiel: Kapselung des Triebwerks	Das Engine -System könnte interne Parameter wie fuelConsumption , thrustLevel oder temperature privat halten. Öffentliche Methoden wie <code>increaseThrust()</code> und <code>decreaseThrust()</code> könnten verwendet werden, um diese Parameter zu steuern, ohne direkten Zugriff auf die internen Daten zu erlauben.
Private Attribute	Private Attribute sind Variablen, die innerhalb einer Klasse definiert sind und nur durch die Methoden dieser Klasse zugänglich sind. In einem Engine -System wären interne Parameter wie fuelLevel , operatingTemperature und currentThrust privat und nicht direkt von außen zugänglich.
Öffentliche Schnittstellen	Public Methods ermöglichen es, auf die Funktionalität des Objekts zuzugreifen, ohne die internen Details preiszugeben. Beispielsweise könnte das FlightControlSystem eine öffentliche Methode <code>adjustAltitude()</code> anbieten, ohne dass der Benutzer wissen muss, wie die FlyByWire -Technologie im Hintergrund funktioniert.
Getter und Setter	Getter-Methoden bieten kontrollierten Zugriff auf private Attribute, und Setter-Methoden ermöglichen kontrollierte Änderungen. Zum Beispiel könnte das CabinPressureSystem einen <code>getPressureLevel()</code> -Getter und einen <code>setPressureLevel(int level)</code> -Setter bereitstellen, der nur valide Werte akzeptiert, um fehlerhafte Eingaben zu verhindern.
Beispiel: Kapselung des Treibstoffsystems	Im FuelSystem könnten interne Berechnungen und Parameter wie currentFuelLevel und fuelFlowRate privat gehalten werden. Öffentliche Methoden wie <code>refuel()</code> oder <code>getFuelStatus()</code> könnten verwendet werden, um diese Attribute zu steuern und Informationen zu liefern, während Details wie die Berechnung des Tankvorgangs verborgen bleiben.

Kategorie	Beschreibung
Vorteile der Kapselung	Datenintegrität: Durch die Kontrolle des Zugriffs auf interne Daten wird sichergestellt, dass diese Daten nur in einem gültigen Zustand verändert werden. Fehlerprävention: Der direkte Zugriff auf sensible oder kritische Daten ist verhindert, was zu weniger fehleranfälligem Code führt. Erhöhte Wartbarkeit: Änderungen an der internen Implementierung sind weniger störend für andere Teile des Systems, da nur die öffentlichen Schnittstellen betroffen sind. Modularität: Kapselung ermöglicht eine klare Trennung von Verantwortlichkeiten zwischen den Systemkomponenten.
Vermeidung von direkten Manipulationen	In einem FlightControlSystem wird die Manipulation von internen Parametern wie altitude oder speed direkt durch kontrollierte Methoden durchgeführt, um zu verhindern, dass externe Komponenten diese direkt verändern, was die Systemintegrität gefährden könnte.
Beispiel: Kapselung der Avionik	Die Avionics-Systeme des Airbus A350-900 könnten private, systemkritische Parameter wie navigationData oder sensorReadings intern verwalten. Öffentliche Schnittstellen wie <code>getFlightPlan()</code> oder <code>updateweatherData()</code> könnten es dem Cockpit ermöglichen, sicher mit diesen Informationen zu interagieren, ohne Details zur internen Datenverarbeitung preiszugeben.
Zusammenhang mit anderen OOP-Prinzipien	Abstraktion: Kapselung ergänzt die Abstraktion, da es die detaillierte Implementierung innerhalb eines Objekts verbirgt und nur die relevanten Funktionen zur Verfügung stellt. Vererbung: In vererbten Klassen können öffentliche Methoden die Funktionalität einer Basisklasse erweitern oder überschreiben, aber die interne Logik bleibt durch Kapselung geschützt. Polymorphie: Kapselung sorgt dafür, dass polymorphe Objekte durch eine einheitliche Schnittstelle sicher angesprochen werden können, ohne dass die Implementierungsdetails des Objekts bekannt sein müssen.

Kapselung (Encapsulation) in Java

01 | Was ist Kapselung in Java?

- **Frage**

Erklären Sie den Begriff Kapselung in Java.

- **Musterlösung**

Kapselung (Encapsulation) ist ein grundlegendes Konzept der objektorientierten Programmierung (OOP), bei dem der Zugriff auf die internen Details eines Objekts eingeschränkt wird. Dies wird erreicht, indem die Attribute einer Klasse privat (`private`) gemacht und über öffentliche Methoden (`getter` und `setter`) zugänglich gemacht werden. Kapselung schützt die Datenintegrität und ermöglicht eine bessere Kontrolle über den Zugriff auf die Daten.

02 | Warum ist Kapselung wichtig in der OOP?

- **Frage**

Erläutern Sie, warum Kapselung ein wichtiges Konzept in der objektorientierten Programmierung ist.

- **Musterlösung** Kapselung ist wichtig, weil sie:

1. **Datenintegrität schützt:** Durch die Einschränkung des direkten Zugriffs auf Attribute können ungültige oder inkonsistente Zustände verhindert werden.
 2. **Flexibilität erhöht:** Die interne Implementierung einer Klasse kann geändert werden, ohne dass dies Auswirkungen auf den Code hat, der die Klasse verwendet.
 3. **Wiederverwendbarkeit fördert:** Gut gekapselte Klassen sind leichter in anderen Projekten oder Kontexten wiederverwendbar.
 4. **Sicherheit verbessert:** Der Zugriff auf sensible Daten kann kontrolliert werden.
-

03 | Was sind Getter- und Setter-Methoden in Java?

- **Frage**

Erklären Sie den Zweck von Getter- und Setter-Methoden in Java.

- **Musterlösung**

Getter- und Setter-Methoden werden verwendet, um den Zugriff auf die privaten Attribute einer Klasse zu kontrollieren.

04 | Was ist der Unterschied zwischen `public`, `private`, `protected` und `default` in Bezug auf Kapselung?

- **Frage**

Erläutern Sie die Sichtbarkeitsmodifikatoren in Java und ihre Rolle bei der Kapselung.

- **Musterlösung**

Die Sichtbarkeitsmodifikatoren legen fest, von wo aus auf Attribute und Methoden zugegriffen werden kann:

- `public`: Der Zugriff ist von überall aus möglich.
 - `private`: Der Zugriff ist nur innerhalb der Klasse möglich.
 - `protected`: Der Zugriff ist innerhalb der Klasse, in Unterklassen und im gleichen Paket möglich.
 - `default` (**kein Modifikator**): Der Zugriff ist nur innerhalb des gleichen Pakets möglich.
-

05 | Was ist der Unterschied zwischen Kapselung und Datenabstraktion?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Datenabstraktion.

- **Musterlösung**

- **Kapselung:** Bezieht sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität durch Einschränkung des Zugriffs auf Attribute.
 - **Datenabstraktion:** Bezieht sich auf das Verbergen der Implementierungsdetails und das Bereitstellen einer vereinfachten Schnittstelle für die Interaktion mit einem Objekt.
-

06 | Wie wird Kapselung in Java implementiert?

- **Frage**

Beschreiben Sie, wie Kapselung in Java implementiert wird.

- **Musterlösung:** Kapselung wird in Java implementiert, indem:

1. Die Attribute einer Klasse als `private` deklariert werden.
 2. Öffentliche Getter- und Setter-Methoden bereitgestellt werden, um den Zugriff auf die Attribute zu kontrollieren.
-

07 | Was ist der Unterschied zwischen Kapselung und Vererbung?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Vererbung.

- **Musterlösung**

- **Kapselung:** Bezieht sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität.
 - **Vererbung:** Bezieht sich auf das Erben von Eigenschaften und Methoden einer Oberklasse durch eine Unterklasse.
-

08 | Was ist der Unterschied zwischen Kapselung und Polymorphismus?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Polymorphismus.

- **Musterlösung**

- **Kapselung:** Bezieht sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität.

- **Polymorphismus:** Bezieht sich auf die Fähigkeit, dass eine Methode in verschiedenen Klassen unterschiedliche Implementierungen haben kann.
-

09 | Was ist der Unterschied zwischen Kapselung und Abstraktion?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Abstraktion.

- **Musterlösung**

- **Kapselung:** Bezieht sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität.
 - **Abstraktion:** Bezieht sich auf das Verbergen der Implementierungsdetails und das Bereitstellen einer vereinfachten Schnittstelle für die Interaktion mit einem Objekt.
-

10 | Was ist der Unterschied zwischen Kapselung und Datenkapselung?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Datenkapselung.

- **Musterlösung**

- **Kapselung:** Ein allgemeiner Begriff, der sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität bezieht.
 - **Datenkapselung:** Ein spezifischer Aspekt der Kapselung, der sich auf das Verbergen und Schützen der Daten (Attribute) einer Klasse bezieht.
-

11 | Was ist der Unterschied zwischen Kapselung und Information Hiding?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Information Hiding.

- **Musterlösung**

- **Kapselung:** Bezieht sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität.
 - **Information Hiding:** Ein Konzept, das eng mit Kapselung verbunden ist und darauf abzielt, die Implementierungsdetails einer Klasse vor dem Benutzer zu verbergen.
-

12 | Was ist der Unterschied zwischen Kapselung und Zugriffskontrolle?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Zugriffskontrolle.

- **Musterlösung**

- **Kapselung:** Bezieht sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität.
 - **Zugriffskontrolle:** Bezieht sich auf die Mechanismen, die den Zugriff auf Attribute und Methoden einer Klasse regeln (z.B. `public`, `private`, `protected`).
-

13 | Was ist der Unterschied zwischen Kapselung und Datenbindung?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Datenbindung.

- **Musterlösung**

- **Kapselung:** Bezieht sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität.
 - **Datenbindung:** Bezieht sich auf die Verknüpfung von Daten mit Benutzeroberflächenelementen.
-

14 | Was ist der Unterschied zwischen Kapselung und Datenkapselung?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Datenkapselung.

- **Musterlösung**

- **Kapselung:** Ein allgemeiner Begriff, der sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität bezieht.
 - **Datenkapselung:** Ein spezifischer Aspekt der Kapselung, der sich auf das Verbergen und Schützen der Daten (Attribute) einer Klasse bezieht.
-

15 | Was ist der Unterschied zwischen Kapselung und Datenabstraktion?

- **Frage**

Erläutern Sie den Unterschied zwischen Kapselung und Datenabstraktion.

- **Musterlösung**

- **Kapselung:** Bezieht sich auf das Verbergen der internen Details einer Klasse und den Schutz der Datenintegrität.
 - **Datenabstraktion:** Bezieht sich auf das Verbergen der Implementierungsdetails und das Bereitstellen einer vereinfachten Schnittstelle für die Interaktion mit einem Objekt.
-

Generalisierung

Aspekt	Beschreibung
Definition	Generalisierung ist ein Konzept der Objektorientierung, bei dem gemeinsame Attribute und Methoden in einer übergeordneten Klasse (Superklasse) zusammengefasst werden, um Code-Wiederverwendung und Wartbarkeit zu verbessern.
Beziehungstyp	/S-A-Beziehung (Erbt eine Klasse von einer anderen, besteht eine Generalisierungsbeziehung).
Schlüsselwort	<code>extends</code> für Klassen, <code>implements</code> für Interfaces.
Klassenhierarchie	Eine Unterklasse (Subklasse) erbt von einer Oberklasse (Superklasse) und kann geerbte Methoden überschreiben oder erweitern.
Methodenvererbung	Methoden der Superklasse sind in der Subklasse verfügbar, sofern sie nicht <code>private</code> oder <code>static</code> sind.
Attributvererbung	Instanzvariablen der Superklasse werden von der Subklasse übernommen, wenn sie <code>protected</code> oder <code>public</code> sind. <code>private</code> Attribute sind nicht direkt vererbbar.
Konstruktoraufruf	Konstruktor der Superklasse wird mit <code>super()</code> aufgerufen, falls nicht explizit definiert, wird der Standardkonstruktor genutzt.
Überschreiben von Methoden	Methoden können mit <code>@override</code> in der Subklasse überschrieben werden, sofern sie nicht <code>final</code> , <code>private</code> oder <code>static</code> sind.
Abstrakte Klassen	Können mit <code>abstract</code> definiert werden und enthalten abstrakte Methoden (<code>abstract</code>), die in den Subklassen implementiert werden müssen.
Interfaces	Bieten eine alternative Form der Generalisierung, indem sie nur Methodensignaturen vorgeben, die von implementierenden Klassen definiert werden müssen.
Mehrfachvererbung	Java unterstützt keine Mehrfachvererbung von Klassen, aber eine Klasse kann mehrere Interfaces implementieren.
Polymorphie	Objekte einer Subklasse können als Objekte der Superklasse behandelt werden (z. B. <code>Superklasse obj = new Subklasse();</code>).
Downcasting & Upcasting	Upcasting (<code>Subklasse</code> → <code>Superklasse</code>) ist implizit, Downcasting (<code>Superklasse</code> → <code>Subklasse</code>) erfordert einen expliziten Cast und kann eine <code>ClassCastException</code> auslösen.
Schutzmechanismen	<code>final</code> verhindert die Vererbung einer Klasse oder das Überschreiben einer Methode.

Airbus A350-900 | Vererbung

Kategorie	Beschreibung
Definition von Vererbung	Vererbung ist das OOP-Prinzip, bei dem eine Klasse (Subklasse) die Eigenschaften und Methoden einer anderen Klasse (Superklasse) erbt. Dies fördert die Wiederverwendbarkeit von Code und ermöglicht eine hierarchische Struktur von Klassen.
Anwendungsfall (Airbus A350-900)	Im Kontext des Airbus A350-900 könnte die Vererbung verwendet werden, um gemeinsame Eigenschaften und Verhaltensweisen von Systemen wie Engines , FlightControlSystems oder Avionics in einer Basisklasse zu definieren, die von spezifischen Implementierungen der Systeme weitervererbt wird.
Ziel der Vererbung	Wiederverwendbarkeit von Code: Gemeinsame Funktionalitäten müssen nur einmal in einer Basisklasse definiert werden und können von allen Unterklassen genutzt werden. Erweiterbarkeit: Neue Systeme oder Komponenten können durch Erweiterung bestehender Klassen ohne umfangreiche Änderungen am bestehenden Code hinzugefügt werden. Hierarchische Struktur: Vererbung ermöglicht eine organisierte und nachvollziehbare Struktur von Klassen.
Beispiel: Vererbung im Triebwerksystem	Eine Basisklasse Engine könnte allgemeine Eigenschaften und Methoden wie <code>start()</code> , <code>stop()</code> , <code>adjustThrust()</code> definieren. Spezifische Triebwerke wie RollsRoyceTrentXWB oder GE90 erben diese Methoden und erweitern sie um zusätzliche, spezifische Implementierungen oder Parameter.
Superklasse (Superclass)	Die Superklasse enthält die gemeinsamen Eigenschaften und Methoden, die von allen Unterklassen geerbt werden. In einem Airbus A350-900 könnte die Superklasse AircraftSystem allgemeine Methoden wie <code>initialize()</code> oder <code>shutdown()</code> bereitstellen, die von allen spezifischen Systemen wie Engine , Avionics , LandingGear erben.
Unterklasse (Subclass)	Die Unterklasse erweitert die Funktionalität der Superklasse, indem sie zusätzliche Eigenschaften und Methoden hinzufügt oder geerbte Methoden überschreibt. Zum Beispiel könnte eine FlyByWireSystem -Unterklasse die geerbte Methode <code>adjustAltitude()</code> spezifisch für die elektrische Steuerung des Flugzeugs implementieren.
Methodenüberschreibung (Method Overriding)	In einer Unterklasse kann eine geerbte Methode überschrieben werden, um sie an die spezifischen Anforderungen der Unterklasse anzupassen. Zum Beispiel könnte die Methode <code>start()</code> in der Engine -Superklasse allgemein sein, aber in der RollsRoyceTrentXWB -Unterklasse spezifische Parameter für den Startvorgang des Triebwerks enthalten.

Kategorie	Beschreibung
Beispiel: Vererbung im Flugsteuerungssystem	Eine Superklasse FlightControlSystem könnte grundlegende Steuerfunktionen wie <code>adjustAltitude()</code> , <code>changeDirection()</code> und <code>adjustSpeed()</code> definieren. Eine spezialisierte Unterklasse wie FlyByWireSystem könnte diese Methoden anpassen, um eine elektronische Steuerung und Flugmanagement zu realisieren, anstatt eine rein mechanische Steuerung zu nutzen.
Polymorphie durch Vererbung	Durch Vererbung können Objekte der Unterklasse durch eine Referenz der Superklasse behandelt werden. Zum Beispiel könnte eine Liste von AircraftSystem -Referenzen sowohl Engine als auch FlightControlSystem -Objekte enthalten, wodurch polymorphe Methodenaufrufe auf diese Objekte angewendet werden können, ohne die genaue Art des Systems zu kennen.
Beispiel: Vererbung im Kabinensystem	Eine Superklasse CabinSystem könnte grundlegende Funktionen wie <code>adjustTemperature()</code> , <code>monitorAirQuality()</code> und <code>adjustLighting()</code> enthalten. Spezifische Klassen wie BusinessClassCabin und EconomyClassCabin könnten von dieser Superklasse erben und eigene Anpassungen für die jeweiligen Klassenzimmer des Flugzeugs vornehmen.
Vorteile der Vererbung	Wiederverwendbarkeit: Gemeinsame Codebasis in Superklassen fördert die Wiederverwendung von Code in Unterklassen. Erweiterbarkeit: Neue Funktionen können durch Erstellen neuer Unterklassen hinzugefügt werden, ohne den bestehenden Code zu ändern. Hierarchie: Eine klare, hierarchische Struktur hilft, die Beziehungen zwischen verschiedenen Komponenten und Systemen zu verstehen.
Zusammenhang mit anderen OOP-Prinzipien	Abstraktion: Vererbung arbeitet eng mit Abstraktion zusammen, da Superklassen abstrakte Eigenschaften und Methoden definieren können, die in den Unterklassen weiter spezifiziert werden. Kapselung: Durch Vererbung können Informationen und Logik in einer Superklasse kapselt werden, sodass die Unterklassen nur die benötigten Eigenschaften und Methoden erhalten. Polymorphie: Vererbung ermöglicht polymorphe Aufrufe von Methoden, die auf Objekte unterschiedlicher Unterklassen angewendet werden.